

Thermostat Dashboard – Operator Manual

This manual is written for a first-time operator. It explains how the dashboard behaves based strictly on the shipped UI structure and source code.

```
Files this manual is based on: - C:/ChatGPT/ai-bots/Thermostats/scripts/Dashboard.py
- C:/ChatGPT/ai-bots/Thermostats/scripts/dashboard_client.js - Generated page:
ai-bots/Thermostats/Web/dashboard_public.html
```

1. What This Dashboard Is

This dashboard is a browser-based view into a thermostat telemetry dataset. It is generated by a Python script and then operated entirely in the browser via JavaScript.

- **System it monitors:** a thermostat/HVAC dataset (project folder name: Thermostats). The code reads rows from a Google Sheet; the thermostat brand/model is not hard-coded in the source.
- **What problem it solves:** it turns raw rows (timestamps, temperatures, modes, runtime) into a live “front panel” plus charts, and it lets you load prior days as archived snapshots.
- **Who it is for:** an operator who needs to quickly understand what the system is doing (now and over time) without digging through raw data.

Important limitation (from code): the generated dashboard is a **static HTML page**. Clicking buttons does not execute Python on a server; it runs client-side JavaScript only.

2. Mental Model of the System

Think of the dashboard as a **data player** with a cassette deck. A “cassette” is one day of saved data. The deck/transport controls are how you move between days, while the chart cursor is how you move within a day.

- **Current data:** embedded into the HTML as JSON inside `<script id="dashboard-data-inline">`.
- **Archive data:** loaded on demand from `ai-bots/Thermostats/Web/chart hist/<YYYY-MM-DD>.json` via `fetch()`.
- **Time index:** each chart point is an index into arrays like `chartLabels[]`, `actual[]`, `setpoint[]`, etc.
- **Selection pointer:** your hover/pin chooses which index the front-panel cards display.

What the cassette spin means: the JavaScript adds a `playing` class to the cassette deck and sets CSS variables for direction and speed. After the spin timer expires, it loads the selected archive JSON and redraws the dashboard.

3. Layout Overview

The dashboard is made of three main areas in small mode, plus a separate full-screen TV mode.

- **A. Front-panel cards** (#Cards): current values like Building Temp, Set Point, Outside Temp, AC Status, Fan Mode, plus request metadata (Type, Studio, Request Expires) when present in the dataset.
- **B. Usage chart** (#usage - slot): a bar chart of condenser runtime totals for a selected range (7 days / month / year).
- **C. Controls + transport**: chart series toggles, Auto-play toggle, step buttons (<< / >>), and the cassette transport panel.
- **D. TV mode**: a full-screen frame that shows the main history chart with a bottom bar (history selector + tape reader) and a nav overlay.

4. Data Cassette

What happens when a cassette (archive day) is loaded:

- The UI chooses a target archive slug (YYYY-MM-DD) and schedules a tape spin.
- After the spin timer, it calls `loadDashboardData(slug)` which fetches `chart_hist/slug.json`.
- When the JSON arrives, it calls `applyDashboardData(data, slug)` which replaces in-memory `dashboardData` and redraws charts/cards.

Spinning cassette (exact meaning):

The cassette reels spin when the deck has CSS class `playing`. Direction and speed are controlled by CSS variables `--deck-spin-direction` and `--deck-spin-duration`.

How the transport relates to time navigation:

- The transport **moves between days** (archives).
- The step buttons (<< / >>) can move **between points within a day**, and when you step past the first/last point it can wrap to adjacent days.
- Tape animation rules (from code): next day = fast spin; forward jumps = fast-forward then play; backward jumps = rewind then play.

5. Buttons and Controls

This section lists each interactive control visible in the HTML and describes exactly what it does. **Note on Python:** clicks do not execute Python; they execute `dashboard_client.js` in the browser.

5.1 Front-panel link

- `#help-chat-link` ("Chat Help"): opens an external chatbot URL in a new tab. Hidden in TV mode by CSS.

5.2 Usage Chart controls

- `.usage-control[data-range="7d"]`: sets usage range to last 7 days (based on available archive slugs).
- `.usage-control[data-range="month"]`: sets usage range to the current month (UTC month of the active archive slug).
- `.usage-control[data-range="year"]`: aggregates runtime by month for the current year (UTC year of the active archive slug).
- **Click a bar** in the Usage chart: loads that day's archive via `loadDashboardData(slug)`.

5.3 Main chart series controls

- `.chart-control[data-mode="setpoint"]`: show/hide the Set Point series.
- `.chart-control[data-mode="actual"]`: show/hide the Building Temp series.
- `.chart-control[data-mode="outside"]`: show/hide the Outside Temp series.
- `.chart-control[data-mode="cooling"]`: show/hide the AC Status series (Idle/Cooling).
- `.chart-control[data-mode="fan"]`: show/hide the Fan Mode series (Auto/Circulate/On).
- `.chart-control[data-mode="both"]` ("Combined"): toggles all series on/off as a group.

5.4 Auto-play

- `#autoplay-toggle`: toggles auto-play. When enabled, after 60 seconds idle the dashboard cycles highlights and series combinations.
- Auto-play pauses can hide the main chart and show the hands/logo for a timed segment; user interaction (mouse/touch/click) stops auto-play and resets the idle timer.

5.5 Step controls (within-day navigation)

- `#chart-step-prev (<<)`: step to the previous point. If you step past the first point, it can load the previous day and pin to the last point.
- `#chart-step-next (>>)`: step to the next point. If you step past the last point, it can load the next day and pin to the first point.
- `#chart-step-value`: readout showing the selected time label (formatted) for the current hover/pin.
- **Hold-to-repeat**: step buttons support press-and-hold repeating after a delay (implemented via pointer/keyboard listeners).

5.6 Transport controls (between-day navigation)

- `#transport-step-month / #transport-step-day`: sets whether the day navigation buttons jump by month or by day.
- `#transport-nav`: the script injects Previous/Next archive buttons here (`#archive-prev` and `#archive-next`).
- **Arrow keys**: Left/Right arrow keys perform the same navigation (day/month/hour depending on TV step mode).
- **Transport panel click** (`#transport-panel`): toggles the history list open/closed. If a history day is “armed” and you click directly on the deck (`#transport-deck`), it loads that selection after the tape animation.

6. Click and Double-Click Behavior

6.1 Single-click (chart point selection)

- Move mouse over the main chart: if nothing is pinned, the dashboard previews (“Hover: ...”) and updates the front-panel cards.
- Click a point on the main chart: pins that point (front-panel cards lock to it). Click the same point again to unpin.
- Click empty chart area: clears pin and suppresses hover updates until you move the mouse out and back in.
- “Clear Pin” button: appears only when a point is pinned; clicking it clears the pin.

6.2 Double-click (enter/exit TV mode)

- Double-click the main chart OR the usage chart: enters TV mode (full-screen frame).
- In TV mode: double-click again to exit back to the dashboard.
- In TV mode: pressing **Escape** also exits.

6.3 State preserved when switching views

- Pinned point selection and active series modes are saved to `localStorage` key `thermostatDashboard.ui.v1`.
- When a new archive loads, the dashboard attempts to restore the pinned selection by matching the saved hour label to the new day’s labels.
- TV mode remembers whether charts were swapped and whether the big chart was hidden, then restores those states on exit.

7. Display Modes

7.1 Small dashboard mode

Small mode shows the front-panel cards, the Usage chart, and the control/transport area. The large TV frame is hidden by the `hide-big-chart` body class.

7.2 Big TV / full-screen mode

- TV mode is enabled by adding `tv-mode` to `document.body`.
- Most small-mode elements are hidden in TV mode (cards, controls block, etc.) by CSS.
- A bottom bar shows the active history day and a tape reader deck; chart controls appear as an overlay.
- If you move the mouse near the bottom edge, the TV controls hint appears (auto-hidden).

7.3 How to enter/exit

- Enter: double-click either chart.
- Exit: double-click in TV mode, or press **Escape**.

8. Form Behavior (What Updates and What Stays)

There is no traditional form submission. The dashboard is a single-page UI with state held in memory and in `localStorage`.

- **Updates dynamically:** front-panel card values, condenser runtime totals, series visibility, tooltips, and selection readouts.
- **Does not reset automatically:** your chosen series modes and pin selection (persisted in `localStorage`).
- **Resets when an archive loads:** chart datasets and labels are replaced by the archive JSON; the UI then tries to re-apply your saved pin/modes.

9. Chatbot Interaction

From the dashboard source code, the chatbot is accessed via the “Chat Help” link and is not embedded into the dashboard state.

What the chatbot is used for:

- Explaining what the user is seeing (cards, charts, controls).
- Helping interpret values and trends (e.g., why the condenser runtime is high).

Allowed actions (based on dashboard code):

- It can open and display guidance (like PDFs) and answer questions.
- It cannot directly change thermostat setpoints or modes through this dashboard, because the dashboard has no API endpoints and runs as static HTML.
- It can instruct the user to click dashboard controls (series toggles, history navigation, TV mode) to change what is displayed.

State-aware suggestions (what it can base suggestions on): the dashboard stores UI state locally (pinned hour and active series). Unless the chatbot has separate access to that browser state, it will not automatically know it; a user may need to describe what they selected.

10. Function Reference (Critical Section)

This section lists Python functions defined in the generator script and what triggers them. For UI behaviors, see the JavaScript functions in `dashboard_client.js`.

46 Python functions were found in the generator script.

`_baseline_anchor_temps(target: date, *, full_sun: bool) -> dict[int, float]`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_classify_condition_from_weather(flags: List[str], precip: List[float]) -> str`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_col_to_index(col: str) -> int`

No docstring in source.

- Called by: `_ensure_range_includes`
- Outputs: returns a value

`_daterange_inclusive(start: date, end: date) -> Iterable[date]`

No docstring in source.

- Called by: `run_projected_range`
- Outputs: no explicit return value (None)

`_day_condition(target: date) -> str`

Deterministic day condition from date only (no external/weather dependencies). Returns: "full_sun", "mixed", or "rainy".

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_daylight_char_for_hour(hour: int) -> str`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_ensure_range_includes(range_value: str, column: str) -> str`

No docstring in source.

- Called by: (no other Python function references found)
- Outputs: returns a value

`_extract_letters(cell: str) -> str`

No docstring in source.

- Called by: `_ensure_range_includes`

- Outputs: returns a value

`_extract_open_meteo_day_hourly(payload: dict, target_day: date) -> tuple[List[float], List[int], List[int], List[float]]`

No docstring in source.

- Called by: `_fetch_open_meteo_hourly_for_day`
- Outputs: returns a value

`_fan_mode_for_hour(target: date, hour: int) -> str`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_fetch_open_meteo_hourly_for_day(target_day: date, tz: str, lat: float, lon: float) -> tuple[List[float], List[int], List[int], List[float]]`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_filter_hourly_rows(rows: List[List[str]], step: int = 12) -> List[List[str]]`

Keep roughly hourly samples (every `step` rows) plus the latest row.

- Called by: (no other Python function references found)
- Outputs: returns a value

`_find_index_in_headers(headers: List[str], keywords: Tuple[str, ...]) -> Optional[int]`

No docstring in source.

- Called by: `_reconstruct_history_rows_from_payload`
- Outputs: returns a value

`_fmt_open_meteo_outside_raw(flag: str, dayc: str, temp_f: float) -> str`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_format_archive_label(slug: str) -> str`

No docstring in source.

- Called by: `build_dashboard_html`, `run_projected_range`
- Outputs: returns a value

`_hourly_drop_for_condition(condition: str) -> float`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_http_get_json(url: str, params: dict) -> dict`

No docstring in source.

- Called by: `_fetch_open_meteo_hourly_for_day`
- Outputs: returns a value

`_interpolate_hourly_from_anchors(anchors: dict[int, float]) -> List[float]`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_load_credentials() -> Credentials`

No docstring in source.

- Called by: `get_drive_service`, `get_sheets_service`
- Outputs: returns a value

`_load_dash_config() -> dict`

Load sheet config from Thermostats config.json, then bot-assets/config.json, then defaults.

- Called by: (no other Python function references found)
- Outputs: returns a value

`_load_drive_credentials() -> Credentials`

Load user OAuth credentials from shared Tokens.json for Drive uploads.

- Called by: `get_drive_upload_service`
- Outputs: returns a value

`_load_float(value, default: float) -> float`

No docstring in source.

- Called by: (no other Python function references found)
- Outputs: returns a value

`_load_global_root() -> Path`

No docstring in source.

- Called by: (no other Python function references found)
- Outputs: returns a value

`_load_shared_config() -> dict`

No docstring in source.

- Called by: (no other Python function references found)
- Outputs: returns a value

`_next_version_name(desired: str, existing_names: List[str]) -> str`

No docstring in source.

- Called by: (no other Python function references found)
- Outputs: returns a value

`_normalize_flag(value, default = True) -> bool`

No docstring in source.

- Called by: (no other Python function references found)
- Outputs: returns a value

`_open_meteo_day_char(is_day_flag: int) -> str`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_open_meteo_flag(code: int, is_day_flag: int) -> str`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`_parse_date_only(value: str) -> date`

No docstring in source.

- Called by: `main`
- Outputs: returns a value

`_reconstruct_history_rows_from_payload(payload: dict) -> Tuple[List[str], List[str], List[List[str]]]`

No docstring in source.

- Called by: `run_test_mode_dashboard`
- Outputs: returns a value

`_setpoint_for_hour(target: date, hour: int) -> float`

No docstring in source.

- Called by: `build_projected_dashboard_payload`
- Outputs: returns a value

`build_dashboard_html(headers: List[str], latest_row: List[str], history_rows: List[List[str]], output_path: Path) -> str`

No docstring in source.

- Called by: `run_live_dashboard`, `run_test_mode_dashboard`
- Outputs: returns a value

`build_projected_dashboard_payload(target: date, archive_dates: List[dict], *, weather_source: str = 'auto', simulate_requests: bool = False, request_every_days: int = 3, request_setpoint_delta_f: float = 2.0, request_setpoint_deltas_f: Optional[List[float]] = None, request_deltas_mode: str = 'random') -> tuple[dict, dict]`

Create a full dashboard payload JSON for the archive viewer, generated from date only. Returns (payload, summary).

- Called by: `run_projected_range`
- Outputs: returns a value

`fetch_sheet_data() -> Tuple[List[str], List[str], List[List[str]]]`

Return headers, latest row, and history rows.

- Called by: run_live_dashboard
- Outputs: returns a value

get_drive_service()

No docstring in source.

- Called by: resolve_sheet_id
- Outputs: returns a value

get_drive_upload_service()

No docstring in source.

- Called by: push_dashboard_to_drive
- Outputs: returns a value

get_or_create_drive_folder(service, folder_name: str) -> str

No docstring in source.

- Called by: push_dashboard_to_drive
- Outputs: returns a value

get_sheets_service()

No docstring in source.

- Called by: fetch_sheet_data
- Outputs: returns a value

git_autopush(html_path: Path, extra_paths: Optional[List[Path]] = None) -> None

Stage, commit, and push the generated dashboard copy.

- Called by: run_live_dashboard
- Outputs: no explicit return value (None)

main(argv: Optional[List[str]] = None) -> None

No docstring in source.

- Trigger: command line entry point (called under __main__).
- Called by: (no other Python function references found)
- Outputs: no explicit return value (None)

push_dashboard_to_drive(html_path: Path, timestamp_suffix: Optional[str] = None) -> str

No docstring in source.

- Called by: run_live_dashboard
- Outputs: returns a value

resolve_sheet_id() -> str

Prefer explicit ID; otherwise resolve by file name.

- Called by: fetch_sheet_data
- Outputs: returns a value

run_live_dashboard() -> None

No docstring in source.

- Called by: main
- Outputs: no explicit return value (None)

run_projected_range(start: date, end: date, *, weather_source: str = 'auto', simulate_requests: bool = False, request_every_days: int = 3, request_setpoint_delta_f: float = 2.0, request_setpoint_deltas_f: Optional[List[float]] = None, request_deltas_mode: str = 'random') -> List[dict]

Generate/overwrite archive JSON datasets for each day in [start, end]. Returns the per-day summaries printed during generation.

- Called by: main
- Outputs: returns a value

run_test_mode_dashboard() -> None

No docstring in source.

- Called by: run_live_dashboard
- Outputs: no explicit return value (None)

upload_or_version_file(service, folder_id: str, filename: str, content: bytes, mime_type: str) -> Tuple[str, str]

No docstring in source.

- Called by: push_dashboard_to_drive
- Outputs: returns a value

11. Typical Usage Flow

11.1 First launch (operator)

- Open the dashboard URL (static page). The page loads embedded JSON and initializes the UI.
- If you do nothing, Auto-play may start after an idle delay and cycle visible series/points.
- If you want manual control, click anywhere on the page or move the mouse to stop Auto-play.

11.2 Read the current state

- Look at the front-panel cards to see the current (or selected) Building Temp, Set Point, Outside Temp, AC Status, Fan Mode, and condenser runtime.
- Use the series buttons to simplify the display (e.g., turn off everything except Set Point).
- Hover the chart (when visible) to preview a moment in time; click to pin if you want to lock the cards to that moment.

11.3 Navigate history (day by day)

- Use the transport navigation (injected Previous/Next buttons) or Left/Right arrow keys to move across archives.
- Watch for the tape spin animation; after it finishes, the dashboard loads the selected day's JSON and redraws.
- Use << / >> step buttons to move within the day; stepping past the edges can automatically wrap to adjacent days.

11.4 Enter TV mode (big view)

- Double-click either chart to enter TV mode.
- Use the TV step mode (Month/Day/Time) and the big << / >> buttons to navigate.
- Exit TV mode by pressing Escape or double-clicking again.